Guía 1 · Construir la app en GitHub, historia por historia

Tablero de Indicadores · Panadería San Nicolás — repositorio `jpablovlsqz16/Proyecto-de-aula-gerencia-` (estructura Frontend / Backend / Docs)

Resultado	El historial de Git muestra la app creciendo desde cero hasta el prototipo final: 16 commits, uno por historia de usuario, cada uno hecho por su responsable en la fecha real.
Material	Carpeta **paquetes/**: una subcarpeta por historia (`HU-01-…` a `HU-16-…`) con los archivos que esa historia crea o cambia y un `COMMIT.txt` con los comandos exactos.
Necesitan	Computador con Git, Cursor (o VS Code) y Node.js 18+; cuenta de GitHub agregada como colaborador del repositorio.
Regla de oro	Los paquetes se aplican **en orden**. Cada historia se integra a `main` antes de empezar la siguiente.

1. Por qué se hace así

El docente observó que el código no estaba en el repositorio y que no había commits en las fechas de los sprints. Con este método cada integrante sube **su** parte, en **su** computador, el día en que le corresponde. Las fechas de los commits son las reales: **nunca** se cambian fechas ni se reescribe el historial.

Cada paquete trae **archivos completos**, no fragmentos. Cuando una historia cambia un archivo que ya existía (por ejemplo `Frontend/js/app.js`), el paquete trae la versión nueva completa y se reemplaza la anterior. Por eso el orden importa.

2. Cronograma de commits

HU	Qué agrega	Responsable	Fecha	Rama
HU-01	estructura de la app, navegación y base de publicación en Apps Script	Iván Santiago	mar 27 oct	`hu-01-estructura-y-arquitectura`
HU-02	plantilla de Excel y diccionario de datos	Juan José	mar 27 oct	`hu-02-plantilla-y-diccionario`
HU-03	simulación de Montecarlo de septiembre de 2026	María Camila	mar 27 oct	`hu-03-datos-simulados`
HU-04	leer el libro de Excel en el navegador	Juan José	mar 27 oct	`hu-04-leer-excel`

HU-05	reconocer las tablas por sus columnas y validar filas	Iván Santiago	mar 27 oct	`hu-05-reconocer-y-validar-tablas`
HU-06	OEE del horno — prototipo v0 publicado en Apps Script	Iván Santiago	mar 27 oct	`hu-06-oee-horno-prototipo-v0`
HU-07	SLA de pedidos (OTIF)	Juan José	mié 28 oct	`hu-07-sla-pedidos`
HU-08	% de producto no vendido	Juan José	mié 28 oct	`hu-08-porcentaje-no-vendido`
HU-09	productividad de mano de obra	Juan José	mié 28 oct	`hu-09-productividad-mano-obra`
HU-10	semáforos y metas de las fichas técnicas	Juan José	mié 28 oct	`hu-10-semaforos-y-metas`
HU-11	filtros por fechas, turno y equipo	Iván Santiago	mié 28 oct	`hu-11-filtros`
HU-12	gráficos del tablero	Iván Santiago	jue 29 oct	`hu-12-graficos`
HU-13	hallazgos automáticos	Juan José	jue 29 oct	`hu-13-hallazgos`
HU-14	exportar CSV e imprimir el tablero	Juan José	jue 29 oct	`hu-14-exportar-e-imprimir`
HU-15	plan de pruebas y verificación con un archivo distinto (octubre)	María Camila	jue 29 oct	`hu-15-pruebas-archivo-distinto`
HU-16	prototipo final publicado en Apps Script y GitHub Pages; manual de usuario	Iván Santiago	jue 29 oct	`hu-16-prototipo-final`

> **Hitos:** al terminar HU-06 queda el **prototipo v0** (OEE del horno) publicado en Google Apps Script; al terminar HU-16, el **prototipo final**. Si un integrante no puede el día asignado, otro lo hace y se anota en el Daily Scrum; lo que no se hace es cambiar la fecha del commit.

3. Preparar el computador (una sola vez, cada integrante)

1. Instalar **Git** (git-scm.com), **Node.js LTS** (nodejs.org) y **Cursor** (cursor.com).
2. Juan Pablo agrega a los demás como colaboradores: GitHub → repositorio → **Settings → Collaborators → Add people**. Cada uno acepta la invitación desde su correo.
3. En Cursor: **Terminal → New Terminal** y escribir (con el nombre y correo de cada uno, el mismo de su cuenta de GitHub):

```
git config --global user.name "Iván Santiago"
git config --global user.email "correo-de-github@ejemplo.com"
git clone https://github.com/jpablovlsqz16/Proyecto-de-aula-gerencia-.git
cd Proyecto-de-aula-gerencia-
```

4. En Cursor: **File → Open Folder** → la carpeta `Proyecto-de-aula-gerencia-`.

> El nombre y el correo de `git config` son los que aparecen como autor del commit. Si el correo no coincide con el de GitHub, el commit no queda asociado al perfil del integrante.

4. Subir una historia (lo mismo para las 16)

Ejemplo con la HU-01. Los comandos exactos de cada historia están en su `COMMIT.txt`.

4.1 Traer lo último de main y crear la rama

```
git checkout main
git pull origin main
git checkout -b hu-01-estructura-y-arquitectura
```

4.2 Copiar los archivos del paquete

1. Abrir la carpeta del paquete (por ejemplo `paquetes/HU-01-estructura-y-arquitectura/`).
2. Copiar **todo su contenido menos COMMIT.txt** y pegarlo en la raíz del repositorio. Cuando Windows pregunte, elegir **Reemplazar los archivos en el destino**.
3. En Cursor, el panel **Source Control** (ícono de ramas) muestra los archivos nuevos (U) y modificados (M). Deben ser exactamente los que lista `COMMIT.txt`.

4.3 Probar

```
npm test        # desde HU-04; en HU-01 a HU-03 basta abrir Frontend/index.html
```

Abrir `Frontend/index.html` con doble clic y revisar lo que la historia agrega (sus criterios de aceptación están en el Word de historias de usuario).

4.4 Commit y push

```
git add -A
git commit -m "feat(HU-01): estructura de la app, navegación y base de publicación en Apps Script"
git push -u origin hu-01-estructura-y-arquitectura
```

4.5 Pull Request y merge

1. En GitHub aparece **Compare & pull request**. Base: `main`. Título: el mismo mensaje del commit. Descripción: «Cumple los criterios de aceptación de la HU-01» y la lista de lo que se probó.
2. **Juan Pablo** (Scrum Master y dueño del repositorio) revisa los archivos y hace **Merge pull request**.

3. **María Camila** (QA) prueba la historia en `main` y mueve la tarjeta en Trello: *En revisión QA* → *Terminado*.
4. Avisar en el grupo: «HU-01 en main». El siguiente responsable empieza desde el paso 4.1.

5. Historias con publicación en Google Apps Script

- **HU-06 (prototipo v0):** después del merge, Iván crea el proyecto en script.google.com, pega `Backend/Code.gs` y `Backend/Index.html` e implementa la aplicación web (Guía 3). La URL `/exec` va al informe.
- **HU-16 (prototipo final):** se pega el nuevo `Backend/Index.html` y se crea una **nueva versión** de la misma implementación: la URL no cambia. Pegar la URL en el README (línea «Aplicación (Google Apps Script)») en un commit aparte: `docs: enlace de la app en Google Apps Script`.
- **GitHub Pages** (HU-16): Settings → Pages → Branch `main`, carpeta `/ (root)` → Save. A los minutos queda en `https://jpablovlsqz16.github.io/Proyecto-de-aula-gerencia-/`.

6. Problemas comunes

Síntoma	Solución
`git push` pide usuario y contraseña o da 403	Aceptar la invitación de colaborador. En Cursor, iniciar sesión con GitHub cuando lo pida (la contraseña ya no sirve; se usa el navegador o un token).
Source Control muestra archivos que no están en COMMIT.txt	Faltó `git pull origin main` o se aplicó un paquete fuera de orden. Descartar los cambios (`git checkout -- .`), volver al paso 4.1 y verificar que la historia anterior ya esté en main.
El Pull Request dice que hay conflictos	Pasa si alguien editó en main los mismos archivos. En la rama: `git pull origin main`, abrir los archivos marcados y quedarse con la versión del paquete; luego `git add -A`, `git commit` y `git push`.
`npm test` falla	No hacer el commit. Revisar que se copiaron todos los archivos del paquete en las carpetas correctas (la carpeta `Tests/` con T mayúscula) y avisar al grupo con el mensaje de error.
`npm` no se reconoce	Instalar Node.js LTS y cerrar y abrir Cursor.

7. Lista de verificación final (29 oct.)

☐ Los 16 Pull Requests están en main, cada uno con el autor y la fecha reales.
☐ `npm test` en main: 33 pruebas aprobadas.
☐ La URL `/exec` de Apps Script abre el prototipo final y genera el tablero con el archivo de octubre.
☐ GitHub Pages abre la misma app.
☐ Trello: las 16 tarjetas en «Terminado» después de la QA.
☐ El informe (Word de historias de usuario y desarrollo) usa la misma numeración que el repositorio.